

UNIVERSIDADE LUSOFONA DO PORTO

ENGENHARIA INFORMÁTICA

Programação de Sistemas Distribuídos

ELETROSYNC

Sistema Distribuído de Comparação de Preços

Desenvolvido por:

Francisco Estrela
João Meneses
Marcelo Campos
Rúben Pereira
Rodrigo Bessa
Tomás Nogueira

Professor: Nuno Coelho

29 de Maio de 2026

Conteúdo

1	Âmbito	3
2	Definição do Problema e Proposta de Solução	3
2.1	Problema	3
2.2	Solução	3
3	Equipa Fundadora / Engenharia	3
4	Requisitos Funcionais	3
4.1	Processamento Concorrente e Organização de APIs	3
4.2	Integração e Extração Multicanal (APIs Rest)	3
4.3	Normalização de dados	4
4.4	Agregação e ordenação instantânea	4
5	Requisitos não funcionais	4
5.1	Performance e Otimização de Latência	4
5.2	Resiliência e Tolerância a Falhas	4
5.3	Escalabilidade Modular	4
5.4	Cache Persistente em Disco	5
6	Requisitos e Experiência de Utilizador	5
6.1	Roadmap (Funcionalidades Futuras)	5
7	Modelo de Negócio e estratégia de Monetização	6
7.1	Marketing de afiliados (B2C)	6
7.2	Listagens Patrocinadas	6
8	Stack Tecnológica e Infraestrutura	7
8.1	Modelo Lógico de Autenticação e Gestão de Identidade	8
9	Documentação da API, Gestão de Cache e Segurança	9
9.1	Endpoints da API e Módulos Lógicos	9
9.1.1	Autenticação e Gestão de Conta	9
9.1.2	Módulo de Favoritos e Carrinho (Requer Autenticação)	9
9.1.3	Módulo de Pedidos (Requer Autenticação)	9
9.1.4	Módulo de Produtos e Integração Externa	10
9.2	Estratégia de Caching Persistente	10
9.3	Segurança e Integridade	10
10	Arquitetura do Sistema	11
10.1	Camada de Clientes	11
10.2	API Gateway / Orquestrador	11
10.3	Camada de Integração Externa (Serper API)	11
10.4	Lojas Externas	12
11	Vantagem Competitiva	12
11.1	Tempo Real vs. Bases de Dados Estáticas	12
11.2	Agilidade de Integração	12

12 Diagrama de Casos de Uso	13
12.1 Identificação dos Atores	14
12.2 Módulos Funcionais e Casos de Uso	14
12.2.1 Gestão de Conta e Autenticação	14
12.2.2 Pesquisa e Decisão de Compra	14
12.2.3 Processamento e Sincronização (Backend)	14
12.2.4 Integração e Monetização B2B	14
13 Descrição do Fluxo de Pesquisa (Padrão Scatter-Gather)	15
14 Conclusão e Trabalho Futuro	17
14.1 Conclusão	17
14.2 Trabalho Futuro	17
15 Interface de Utilizador	18

1 Âmbito

O EletroSync é uma plataforma de agregação e comparação de preços a operar em tempo real, a nossa plataforma vai focar-se numa arquitetura de microserviços/sistemas distribuídos. Numa fase inicial de PoC (Proof of Concept), o sistema vai focar-se só no nicho de mercado de eletrodomésticos e produtos tecnológicos.

2 Definição do Problema e Proposta de Solução

2.1 Problema

A fragmentação de preços nas vendas online obriga os utilizadores a fazer várias pesquisas manuais, num processo que é irritante, lento e sequencial.

2.2 Solução

Uma arquitetura distribuída que vai consultar múltiplas fontes de dados em simultâneo (paralelismo). Isto vai permitir fornecer ao utilizador o preço mais baixo no mercado, ordenado instantaneamente e isso otimiza o período de compra.

3 Equipa Fundadora / Engenharia

A conceção técnica e o desenvolvimento desta arquitetura distribuída são assegurados por: Francisco Estrela, João Meneses, Marcelo Campos, Rúben Pereira, Rodrigo Bessa e Tomás Nogueira.

4 Requisitos Funcionais

Esta secção detalha as capacidades cruciais do sistema EletroSync, focando-se na forma como o Backend interage com os diferentes ecossistemas externos e processa os dados para entregar os respetivos dados ao utilizador final.

4.1 Processamento Concorrente e Organização de APIs

Para se diferenciar das consultas tradicionais o núcleo de agregação do EletroSync assenta no paralelismo. O sistema é capaz de lançar múltiplos pedidos de API em simultâneo para os diversos retalhistas, orquestrando as respostas em tempo real.

4.2 Integração e Extração Multicanal (APIs Rest)

A Plataforma foi desenhada para consumir nativamente as APIs das lojas integradas. A extração incide sobre três pontos de dados mais críticos para a conversão nas vendas:

- O Preço atualizado
- Disponibilidade de stock
- Imagem representativa do produto

4.3 Normalização de dados

Sendo um agregador, o sistema lida de forma inata com a heterogeneidade de payloads. O EletroSync tem a capacidade de receber as respostas em múltiplos e diferentes formatos, aplicando uma camada de limpeza. Embora a utilização de Serper ajuste parcialmente a camada de rede ao entregar um JSON com estrutura unificada, os dados extraídos de cada loja continuam a apresentar uma elevada heterogeneidade no conteúdo.

O EletroSync garante a normalização avançada destes dados (formatação de preços, mapear metadados de stock, validade URLs de imagem), garantindo uniformidade independentemente da formação específica que tenha sido aplicada pelo retalhista na sua origem.

4.4 Agregação e ordenação instantânea

Todo o pipeline de dados terá capacidade de processar a informação recolhida e apresentar imediatamente ao utilizador o preço mais baixo disponível no mercado ordenado e filtrado.

5 Requisitos não funcionais

Esta secção define os critérios de qualidade, performance e resiliência da arquitetura do EletroSync, garantindo que o sistema está preparado para operar num ambiente de produção exigente.

5.1 Performance e Otimização de Latência

O sistema garante uma latência mínima. Como as procuras são executadas simultaneamente através de processos concorrentes, o tempo total de espera do utilizador corresponde estritamente ao tempo de resposta da loja mais lenta, não sendo cumulativo.

5.2 Resiliência e Tolerância a Falhas

A arquitetura isola falhas de terceiros: se a API de um retalhista (ou o próprio Serper) demorar ou falhar, esse fetch é abortado via AbortController e o sistema continua com as restantes lojas. Outros mecanismos de resiliência:

- **Timeouts** agressivos em todos os fetches (5-8s no Serper, 60s no auto-refresh interno)
- **Mutex no /all** evita thundering herd quando vários utilizadores chegam com cache fria (1 chamada Serper em vez de $N \times 14$)
- **Graceful shutdown** (SIGTERM/SIGINT) com flush síncrono da cache para não perder writes pendentes do debounce
- **Cache size cap** de 5 MB com reset automático no arranque

5.3 Escalabilidade Modular

O sistema está desenhado para uma escalabilidade horizontal simples. A integração de futuros retalhistas no ecossistema requer apenas a adição de um novo módulo de API à camada de Workers, sem necessidade de refatorização ou impacto no código do sistema central.

5.4 Cache Persistente em Disco

Para mitigar chamadas redundantes à API Serper e acelerar respostas, o sistema implementa uma camada de cache **persistente em disco** sob a forma de ficheiros JSON (`cache/serper-cache.json` e `cache/store-urls.json`). A cache sobrevive a reinícios do servidor.

Time-To-Live (TTL) por tipo de dado:

- **Catálogo base (/all):** 12 horas
- **Resultados de pesquisa (/search):** 4 horas
- **Comparação de preços (/compare):** 2 horas
- **URLs de produto resolvidos:** 7 dias
- **Preços live (fetch HTML Rádio Popular):** 2 horas
- **Imagens PNG do hero rotativo:** 7 dias

Garbage collection automático no arranque remove entradas expiradas. Auto-refresh agendado limpa entradas de produto/preço duas vezes por dia, garantindo que os preços nunca ficam mais de ~12h desatualizados.

6 Requisitos e Experiência de Utilizador

Funcionalidades implementadas na versão atual:

- **Gestão de Identidade:** registo e login por email/password com confirmação por email; recuperação de palavra-passe via link; edição de dados pessoais (nome, email com confirmação) e palavra-passe; eliminação permanente de conta
- **Lista de Favoritos:** utilizadores autenticados podem guardar produtos para monitorizar flutuações de preço
- **Carrinho Inteligente:** agrega produtos selecionados e calcula automaticamente a *melhor loja* (loja onde o total do carrinho é mais barato)
- **Checkout Simulado** + Histórico de Pedidos para registo de compras
- **Pesquisa com Autocomplete:** dropdown em tempo real com produtos sugeridos enquanto o utilizador escreve
- **Filtros e Ordenação:** por loja, intervalo de preço, categoria, subcategoria; ordenação por relevância, preço (asc/desc) ou nome
- **Mobile Menu** responsivo com hamburguer e drawer lateral
- **Toasts de Feedback** para ações de carrinho/favoritos/auth

6.1 Roadmap (Funcionalidades Futuras)

- Autenticação via Google (Supabase OAuth).
- Motor de alertas de preço com notificações.
- Histórico de evolução de preços por produto.
- Segurança Multi-Fator (MFA).

7 Modelo de Negócio e estratégia de Monetização

O EletroSync assenta num modelo híbrido e escalável, concebido para diversificar fontes de receita, operando em duas frentes principais, B2C (business to consumer) e B2B (business to business). A sustentabilidade financeira da plataforma vai permitir que o serviço se mantenha 100% gratuito e sem barreiras de entrada.

7.1 Marketing de afiliados (B2C)

Por cada redirecionamento do EletroSync que resulte numa compra concretizada numa loja externa, a plataforma retém uma comissão percentual, que no setor da tecnologia e eletrodomésticos varia tipicamente entre 1% a 5% do valor do carrinho.

7.2 Listagens Patrocinadas

À medida que o volume da plataforma EletroSync aumenta, o EletroSync abrirá um espaço para Featured Products, ou seja, os retalhistas poderão licitar para destacar produtos ou campanhas específicas na plataforma, sempre com a devida sinalização de Patrocínio para transparecer confiança.

8 Stack Tecnológica e Infraestrutura

Para materializar a arquitetura proposta e garantir a eficiência exigida pela Prova de Conceito (PoC), o EletroSync foi desenvolvido com recurso a um ecossistema de tecnologias modernas, focadas na alta performance, segurança e rápida escalabilidade. A stack selecionada divide-se nos seguintes componentes fundamentais:

- **Backend e Organização: Node.js + Express**

A escolha do ambiente Node.js foi estratégica. A sua arquitetura assíncrona e orientada a eventos é a ferramenta perfeita para implementar o paralelismo exigido pelo padrão Scatter-Gather. O framework Express foi utilizado para construir a API RESTful de forma leve e minimalista, garantindo o encaminhamento rápido dos pedidos entre o cliente e os Workers.

- **Frontend e Interface: HTML/CSS/JS + Bootstrap 5**

Para a camada de apresentação, optou-se por tecnologias web nativas (JavaScript) combinadas com a grelha e componentes do Bootstrap 5. Esta abordagem garante uma interface responsiva, rápida a carregar e isenta da sobrecarga de frameworks mais complexos nesta fase inicial do projeto.

- **Base de Dados e Identidade: Supabase (PostgreSQL + RLS)**

O Supabase atua como o motor principal de persistência e gestão de acessos. Assentando numa base de dados relacional robusta (PostgreSQL), permite o armazenamento fiável de perfis e histórico de pesquisas. O sistema de Autenticação integrado e a utilização de Row Level Security (RLS) garantem um isolamento rigoroso dos dados: cada utilizador tem apenas permissão de acesso à sua própria área de favoritos, carrinho e alertas, cumprindo os melhores padrões de cibersegurança.

- **Agregação de Dados: Serper API (Google Shopping)**

Para resolver a volatilidade típica dos web scrapers tradicionais, o backend integra a API do Serper. Esta ferramenta atua como o motor de pesquisa distribuída, extraíndo informação estruturada (JSON) em tempo real através do Google Shopping, garantindo fiabilidade e padronização na recolha de preços e stock.

- **Infraestrutura e Deployment: Render.com**

O alojamento do EletroSync é assegurado pela plataforma Render. Operando num modelo Platform-as-a-Service (PaaS), simplifica a Integração e Entrega Contínuas a partir do repositório de código, e garante que a aplicação está pronta para escalar rapidamente os seus recursos perante o aumento de tráfego.

- **Comunicações e Notificações: Resend**

Para a gestão de emails transacionais uma peça fundamental para os fluxos de gestão de identidade foi a API do Resend. Esta plataforma foi escolhida pela sua elevada taxa de entregabilidade, arquitetura moderna focada na experiência do programador e integração nativa simplificada com o ecossistema Node.js, substituindo a complexidade da gestão de servidores SMTP tradicionais por chamadas REST altamente eficientes.

- **Validação Live de Preços: Fetch HTML direto**

Quando o utilizador entra na página de detalhe de um produto, o sistema faz um fetch HTML direto à página da Rádio Popular para garantir que o preço exibido é 100% fiel ao valor atualizado. As restantes lojas (como Worten e FNAC), devido a proteções Cloudflare, mantêm a extração primária via Serper Shopping.

8.1 Modelo Lógico de Autenticação e Gestão de Identidade

Para sustentar o rigoroso controlo de acessos e a segurança dos dados da plataforma, o EletroSync apoia-se no esquema nativo de autenticação do Supabase. A Figura ilustra o Diagrama de Entidade-Relação (ER) simplificado deste módulo central.

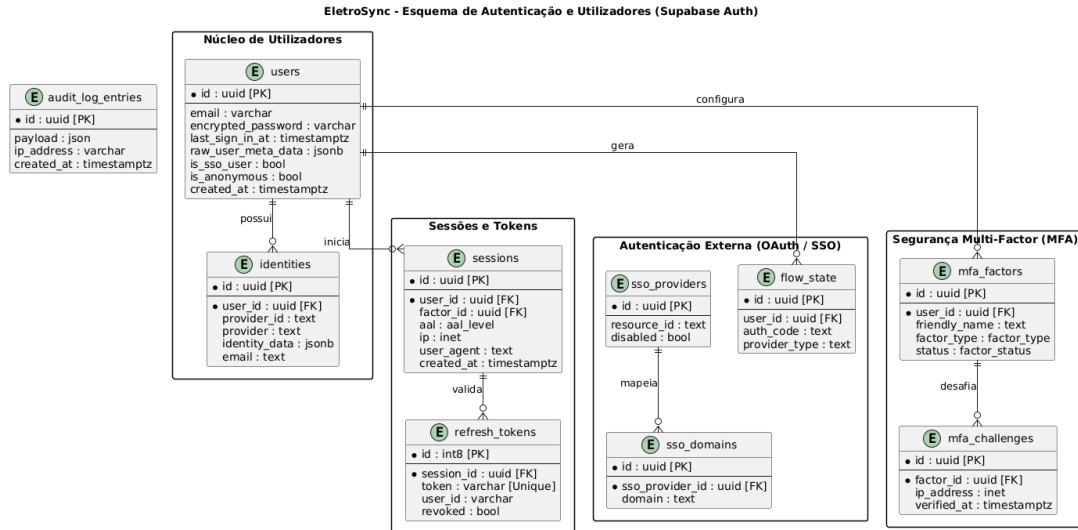


Figura 1: Diagrama de Entidade-Relação (ER) do módulo de Autenticação (Supabase)

A arquitetura relacional deste esquema divide-se em quatro pilares fundamentais de segurança:

- **Núcleo de Utilizadores:** A tabela principal `users` centraliza a identidade e os metadados do cliente. Através da relação com a tabela `identities`, o sistema permite que um único utilizador possua múltiplos métodos de login (ex: e-mail tradicional e Single Sign-On via Google) unificados na mesma conta.
- **Gestão de Sessões e JWT:** Para garantir uma experiência contínua sem comprometer a segurança, as instâncias de login ativas são registadas na tabela `sessions`. A tabela `refresh_tokens` assegura a rotação segura das credenciais de acesso de curta duração (JWT - JSON Web Tokens).
- **Autenticação Externa (OAuth/SSO):** As tabelas `flow_state` e `sso_providers` gerem o estado transacional quando o sistema delega a verificação de identidade a fornecedores externos, garantindo que o fluxo de autenticação não sofra interceções (Man-in-the-Middle).
- **Segurança Multi-Fator (MFA):** A estrutura contempla nativamente tabelas como `mfa_factors` e `mfa_challenges`, deixando a infraestrutura do EletroSync totalmente preparada para escalar os requisitos de segurança, permitindo exigir um segundo passo de verificação no futuro.

O aproveitamento deste modelo rigoroso em PostgreSQL permite ao EletroSync delegar a complexidade criptográfica e a gestão de tokens a uma infraestrutura testada, focando o esforço de desenvolvimento na lógica de negócio da agregação de preços.

9 Documentação da API, Gestão de Cache e Segurança

Para garantir a comunicação eficiente entre a camada de apresentação e os serviços de backend, foi desenvolvida uma API RESTful estruturada. A arquitetura contempla políticas rigorosas de Rate Limiting, caching dinâmico e segurança aplicacional.

9.1 Endpoints da API e Módulos Lógicos

A API encontra-se segmentada em módulos, garantindo o isolamento de rotas públicas e rotas protegidas (que exigem validação de identidade via Bearer token).

9.1.1 Autenticação e Gestão de Conta

Método	Path	Descrição
POST	/api/auth/register	Cria conta nova (envia email de confirmação).
POST	/api/auth/login	Autentica e devolve tokens.
POST	/api/auth/logout	Termina a sessão local.
POST	/api/auth/forgot-password	Envia email com link de recuperação.
POST	/api/auth/update-password	Define nova password via token.
PUT	/api/auth/profile	Atualiza nome e/ou email do utilizador.
POST	/api/auth/change-password	Altera password (re-verifica a antiga).
DELETE	/api/auth/account	Elimina permanentemente a conta + dados.

9.1.2 Módulo de Favoritos e Carrinho (Requer Autenticação)

Método	Path	Descrição
GET	/api/favorites	Lista os favoritos do utilizador autenticado.
POST	/api/favorites	Adiciona um novo artigo aos favoritos.
DELETE	/api/favorites/:product_id	Remove um artigo específico dos favoritos.
GET	/api/cart	Lista os itens presentes no carrinho.
POST	/api/cart	Adiciona/incrementa um produto ao carrinho.
PUT	/api/cart/:product_id	Atualiza a quantidade de um item.
DELETE	/api/cart/:product_id	Remove um item específico do carrinho.
DELETE	/api/cart	Limpa a totalidade do carrinho.

9.1.3 Módulo de Pedidos (Requer Autenticação)

Método	Path	Descrição
GET	/api/orders	Lista pedidos do utilizador (recentes primeiro).
GET	/api/orders/:id	Detalhe de um pedido.
POST	/api/orders	Cria pedido a partir do carrinho + envio.

9.1.4 Módulo de Produtos e Integração Externa

De forma a proteger a infraestrutura e os custos da API agregadora (Serper), as rotas de produtos implementam limites de requisição (Rate Limiting).

Método	Path	Limite	Descrição
GET	/api/products/all	60/min	Retorna o catálogo base em cache.
GET	/api/products/search	20/min	Pesquisa dinâmica com aplicação de filtros.
GET	/api/products/compare	20/min	Compara preços em simultâneo nas várias lojas.
GET	/api/products/details	20/min	Devolve especificações técnicas do produto.
GET	/api/products/hero-images	60/min	PNGs para hero (cached 7 dias).
GET	/api/products/refresh	10/h	Limpeza manual de cache.
GET	/api/products/cache-status	60/min	Diagnóstico do estado da cache do sistema.

9.2 Estratégia de Caching Persistente

- **Catálogo Base (12 horas):** Produtos estáveis e metadados gerais.
- **Resultados de Pesquisa (4 horas):** Um equilíbrio ideal para absorver picos de utilizadores a pesquisar pelos mesmos termos ao longo do dia.
- **Comparação de Preços (2 horas):** A janela temporal mais curta, visto que os preços e a disponibilidade de stock sofrem flutuações rápidas nos retalhistas parceiros.
- **URLs de Produto (7 dias):** Os identificadores e links de redirecionamento para as lojas sofrem raríssimas alterações, permitindo uma retenção longa e eficiente.
- **Imagens PNG do hero rotativo (7 dias):** Retenção de longa duração para os ativos visuais gerados via endpoint `/hero-images`.

9.3 Segurança e Integridade

A proteção dos dados dos utilizadores e da infraestrutura é assegurada pelas seguintes medidas:

- **Row Level Security (RLS):** Ativo em todas as tabelas da base de dados Supabase, garantindo criptograficamente que cada utilizador interage apenas com os dados que lhe pertencem.
- **Mitigação de Abusos:** Rate limiting por IP aplicado a todas as rotas que consomem a API Serper.
- **Validação em Middleware:** Todos os endpoints autenticados operam sobre o middleware `requireAuth`, assegurando a validação dos tokens a cada request, aliada à sanitização estrita de todos os inputs.
- **Isolamento de Credenciais:** O ficheiro `.env` encontra-se devidamente ignorado via `.gitignore`, prevenindo fugas de chaves de API e segredos de ambiente para os repositórios de código.

10 Arquitetura do Sistema

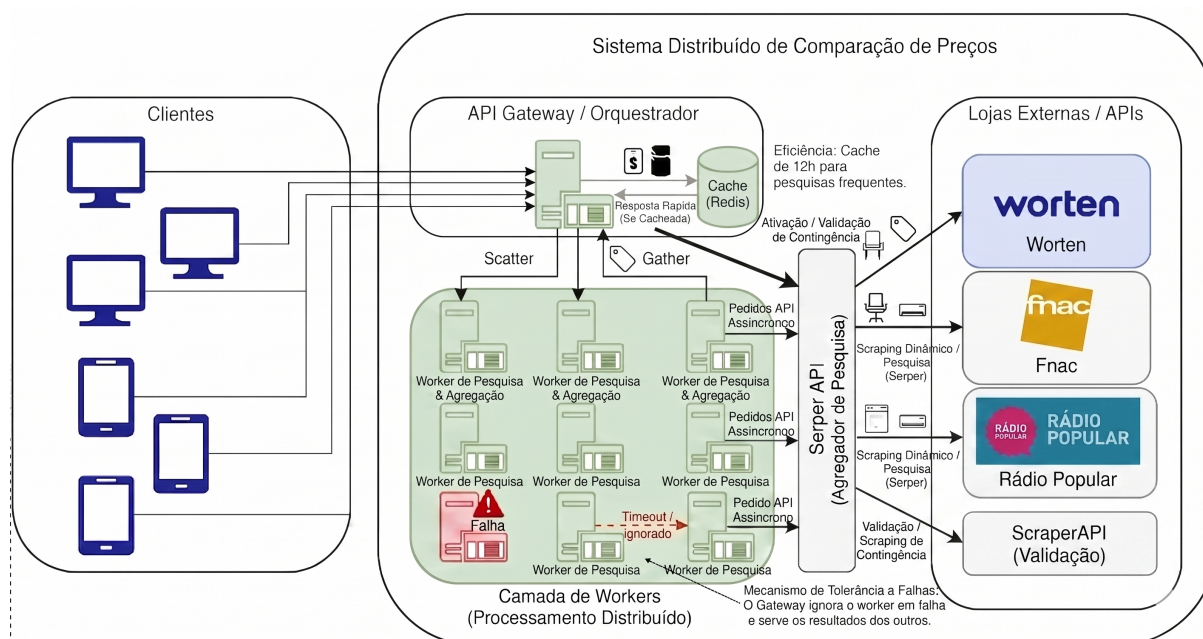


Figura 2: Diagrama de Arquitetura do Sistema Distribuído

A arquitetura do EletroSync foi desenhada com base num modelo de microserviços distribuídos, garantindo alta disponibilidade, baixa latência e resiliência a falhas de terceiros. O fluxo de processamento de um pedido atravessa quatro camadas fundamentais:

10.1 Camada de Clientes

O fluxo inicia-se nos utilizadores finais. Os clientes efetuam um pedido de pesquisa de um produto que é enviado para um ponto único de entrada no nosso backend.

10.2 API Gateway / Orquestrador

É o cérebro do sistema, a cada requisição o orquestrador verifica a cache em primeiro lugar. Se a pesquisa tiver sido feita recentemente, devolve uma resposta rápida com os recursos que ficam guardados em cache durante um intervalo de tempo. Se não houver nada na cache, o orquestrador distribui e divide o pedido por vários Workers independentes.

10.3 Camada de Integração Externa (Serper API)

O Serper atua como proxy estruturado para o Google Shopping/Search/Images, devolvendo JSON normalizado em vez de HTML scrape. Vantagens face a scraping tradicional: não há que lidar com Cloudflare, captchas ou mudanças de DOM nas lojas. Custos: 1 crédito por query (free tier de 2500 créditos/mês).

10.4 Lojas Externas

Lojas verificadas com regex específica de URL de produto (garantindo que o link aberto pelo utilizador é uma página de produto, não categoria/pesquisa):

- **Worten**-worten.pt/produtos/...
- **FNAC**-fnac.pt/<slug>/aNNNNNNN
- **Rádio Popular**-radiopopular.pt/produto/...
- **PC Diga**-pcdiga.com/<cat>/<sub>/...<id>

Nota: A arquitetura permite acrescentar novas lojas (MediaMarkt, Amazon, El Corte Inglés, etc.) adicionando entradas à configuração STORES. Como trabalho futuro, prevê-se integrar pelo menos mais 3 lojas portuguesas.

11 Vantagem Competitiva

Num mercado onde já existem comparadores de preços fixos, o EletroSync diferencia-se fundamentalmente pela sua infraestrutura tecnológica de agregação:

11.1 Tempo Real vs. Bases de Dados Estáticas

A maioria dos comparadores tradicionais depende de catálogos desatualizados ou integrações lentas. O EletroSync diferencia-se pela sua extração on-demand. Embora use uma cache inteligente de 12 horas para otimizar a performance e reduzir custos de API em pesquisas frequentes, a arquitetura distribuída (Serp + Workers) permite que qualquer novo produto pesquisado seja extraído instantaneamente da fonte, garantindo fiabilidade dos preços e agilidade na entrega de resultados em comparação com sistemas de indexação monolíticos. Adicionalmente, a dupla camada de extração (Serper API + fetch HTML direto para a Rádio Popular) assegura uma confiança absoluta no EletroSync. Ao contrário dos sistemas clássicos que deixam os preços desatualizados durante dias, a nossa arquitetura tem a capacidade de se autocorrigir em tempo real no exato momento em que deteta que o preço armazenado não corresponde ao carrinho da loja final.

11.2 Agilidade de Integração

O processamento assíncrono permite que a plataforma adicione novas lojas nacionais ou internacionais numa questão de horas, sem necessidade de reestruturar o sistema central, garantindo uma escalabilidade de mercado sem precedentes. A capacidade de usar serviços de pesquisa distribuída mostra que o sistema suporta a adição de novos domínios de retalhistas quase instantaneamente, sem necessidade de reestruturação do código central.

12 Diagrama de Casos de Uso

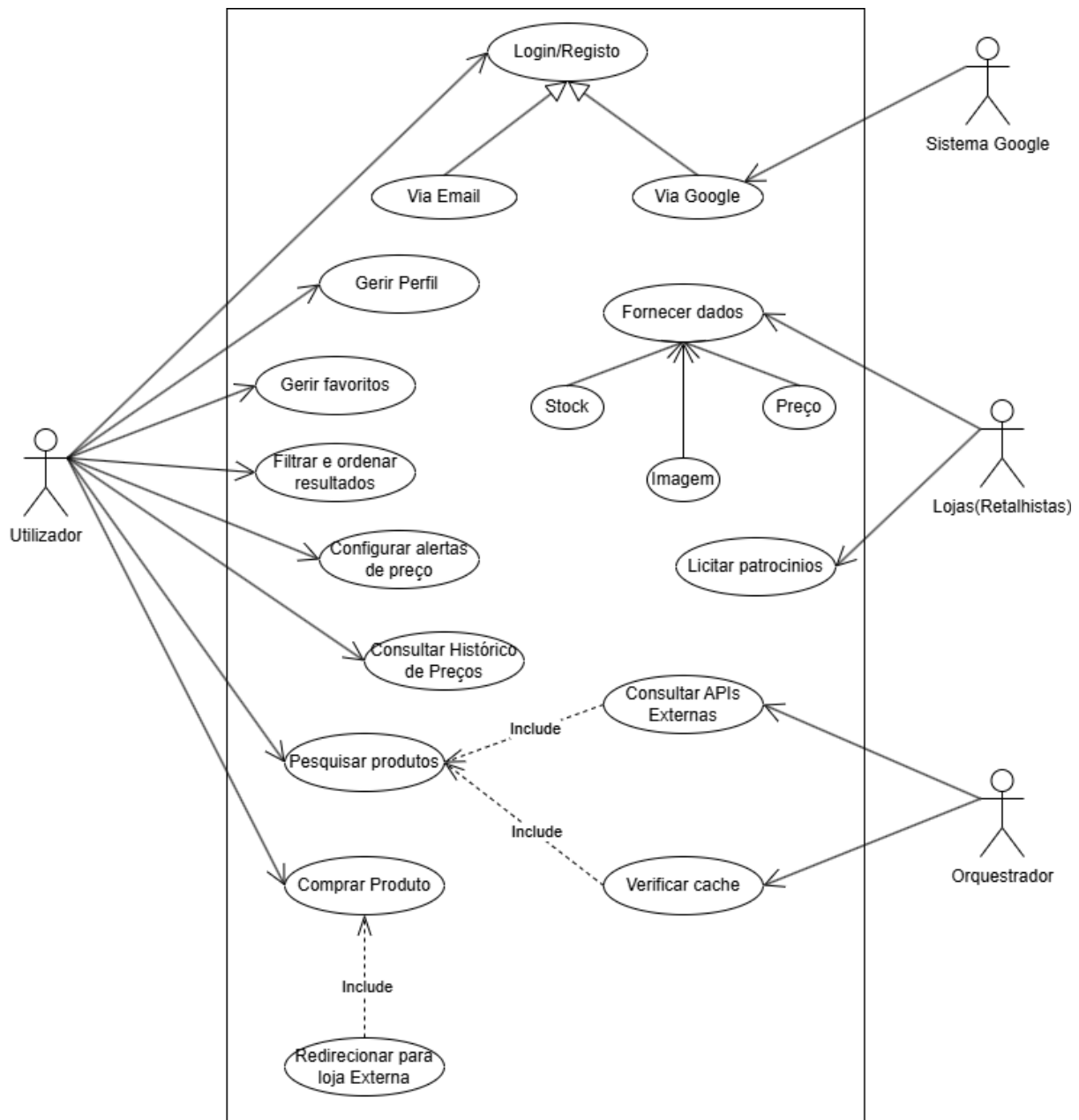


Figura 3: Diagrama Casos de Uso

O diagrama de casos de uso apresentado ilustra as interações funcionais do sistema ElectroSync, mapeando os principais atores envolvidos e as ações que estes podem executar na plataforma. O sistema foi desenhado para suportar tanto a vertente de utilizador final (B2C) como a integração com parceiros comerciais (B2B) e os processos automatizados de backend.

12.1 Identificação dos Atores

O sistema contempla quatro atores principais, divididos entre utilizadores reais e sistemas automatizados:

Utilizador: O ator principal (cliente) que interage ativamente com a interface para pesquisar produtos, analisar preços e gerir a sua conta.

Lojas (Retalhistas): Entidades parceiras externas que fornecem os dados do seu catálogo e interagem com a vertente comercial da plataforma.

Orquestrador: Ator de sistema que atua de forma invisível para o cliente final, responsável pela gestão de tráfego, cache e comunicação concorrente com as APIs.

Sistema Google: Ator de sistema externo utilizado para facilitar a autenticação.

12.2 Módulos Funcionais e Casos de Uso

12.2.1 Gestão de Conta e Autenticação

O Utilizador pode autenticar-se na plataforma através do caso de uso genérico “Login/Registo”. Esta ação possui duas especializações: a autenticação tradicional “Via Email” ou a autenticação “Via Google”. Uma vez autenticado, o Utilizador tem acesso a funcionalidades como “Gerir Perfil” e “Gerir favoritos”.

12.2.2 Pesquisa e Decisão de Compra

O núcleo da plataforma gira em torno do caso de uso “Pesquisar produtos”. Para melhorar a experiência de descoberta e decisão, o Utilizador pode:

- Filtrar e ordenar resultados: Ajustar a listagem consoante preferências específicas.
- Configurar alertas de preço: Definir valores-alvo para ser notificado.
- Consultar Histórico de Preços: Analisar a evolução temporal do valor de um artigo.

Quando o Utilizador decide avançar, passa para o caso de uso “Comprar Produto”, que obrigatoriamente inclui a ação de “Redirecionar para loja Externa”, concretizando assim o modelo de negócio de marketing de afiliados.

12.2.3 Processamento e Sincronização (Backend)

Sempre que uma pesquisa é iniciada, ocorrem processos dependentes cruciais para a arquitetura de baixa latência. O ator Orquestrador entra em ação para gerir estes fluxos:

- Verificar cache: Avaliar se os resultados daquela pesquisa já se encontram armazenados em memória temporária para uma resposta rápida.
- Consultar APIs Externas: Caso não haja cache válida, o Orquestrador comunica com as diferentes lojas em simultâneo.

12.2.4 Integração e Monetização B2B

As Lojas/Retalhistas alimentam a plataforma através do caso de uso genérico de Fornecer dados, que abrange informações detalhadas e especializadas como Stock, Imagem e Preço. Adicionalmente, estes parceiros podem interagir ativamente com o módulo comercial para Licitar patrocínios, garantindo destaque para os seus produtos nas listagens.

13 Descrição do Fluxo de Pesquisa (Padrão Scatter-Gather)

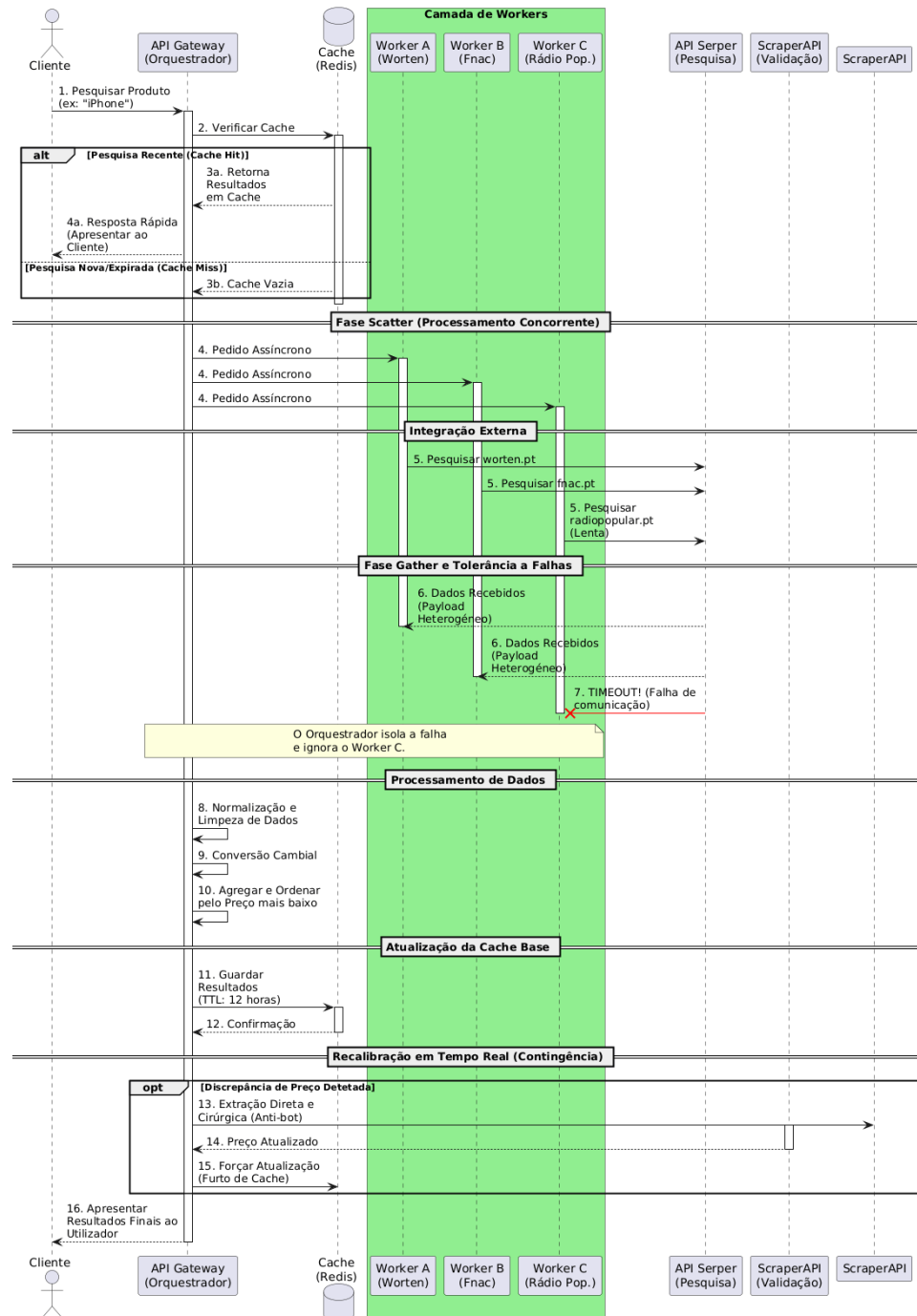


Figura 4: Diagrama de Sequência

O Diagrama de Sequência ilustra o ciclo de vida completo de uma pesquisa no EletroSync, mostrando a arquitetura de baixa latência e a resiliência do sistema a falhas externas. O fluxo divide-se nos seguintes passos críticos:

Início do Pedido: O Cliente submete uma pesquisa por um produto. O pedido entra no sistema através de um ponto único: API Gateway / Orquestrador.

Verificação de Cache: Sendo o cérebro do sistema, a primeira ação do Orquestrador é consultar a base de dados em disco.

Caso de Sucesso: Se a pesquisa foi realizada recentemente, o sistema devolve imediatamente os dados guardados em cache, garantindo uma resposta quase instantânea.

Distribuição Concorrente: Caso não existam dados válidos na cache, o Orquestrador divide o pedido, acionando múltiplos Workers de Pesquisa de forma paralela e independente.

Comunicação Assíncrona: Cada Worker estabelece uma ligação isolada com a API do Serper, segmentando o pedido para o domínio do retalhista correspondente. Este serviço distribuidor faz o scraping dinâmico e extrai o preço, imagem e disponibilidade da loja final.

Recolha e Tolerância a Falhas (Gather): Os Workers recebem as respostas e enviam-nas de volta para o Orquestrador. Neste passo, é ativado o mecanismo de tolerância a falhas: se a API de um retalhista demorar demasiado a responder, o Worker entra em estado de falha e o Orquestrador simplesmente ignora esse nó, avançando com os restantes.

Processamento Final: O Orquestrador aplica a camada de sanitização. Em seguida, processa toda a informação para apresentar o preço mais baixo ordenado de forma instantânea.

Atualização de Cache e Resposta: Antes de devolver os resultados ao Cliente, o Orquestrador guarda o resultado desta extração numa cache persistente em disco JSON com um Time-To-Live (TTL) adaptativo. Durante este período, qualquer utilizador que pesquise pelo mesmo termo receberá instantaneamente a resposta, fechando assim o ciclo.

14 Conclusão e Trabalho Futuro

14.1 Conclusão

O EletroSync valida a viabilidade de uma plataforma de comparação de preços moderna, com baixa latência e alto cache hit rate. A combinação de padrão Scatter-Gather (via Promise.all), cache persistente em disco com TTL diferenciado, e matching canónico via productId do Google Shopping resolve os problemas clássicos de plataformas anteriores: latência alta, dados desatualizados e variantes confusas entre lojas.

A infraestrutura cumpre princípios de resiliência: timeouts em todas as chamadas externas, isolamento de falhas, mutex anti-thundering-herd, graceful shutdown com flush de cache, validação multi-camada de URLs e preços. Esta robustez sustenta o modelo de negócio de afiliados (B2C) e abre caminho às futuras listagens patrocinadas (B2B).

14.2 Trabalho Futuro

Sendo o EletroSync um sistema modular e desenhado para escalar facilmente, a visão a longo prazo para o projeto engloba diversas melhorias tecnológicas e a exploração de novas vertentes de mercado. Numa próxima iteração de desenvolvimento, prevê-se a aposta nos seguintes eixos:

- **Login via Google (OAuth)** – Supabase suporta nativamente; ativação é questão de configurar redirect e adicionar botão na UI
- **Motor de Alertas de Preço** – nova tabela price_alerts, verificação no auto-refresh, notificação por email via Resend
- **Histórico de Preços** – registar snapshot do preço a cada auto-refresh e mostrar gráfico de evolução por produto
- **Mais Lojas Parceiras** – MediaMarkt, PcComponentes, El Corte Inglés, Amazon, integradas via novas entradas em STORES
- **Conversão Cambial** – para suporte multi-moeda em futura expansão ibérica (integração com API exchangerate.host)
- **Multi-Factor Authentication (MFA)** – Supabase tem schema pronto; ativar TOTP/SMS para contas premium
- **Aplicação Mobile** – wrapper React Native ou PWA com scanner de códigos de barras para consulta em loja física
- **Big Data e Analytics (B2B)** – portal de parceiros com métricas de posicionamento, tempos de resposta e tendências de pesquisa
- **Infraestrutura Cloud-Native** – migração para Kubernetes com auto-scaling horizontal quando o tráfego justificar

15 Interface de Utilizador

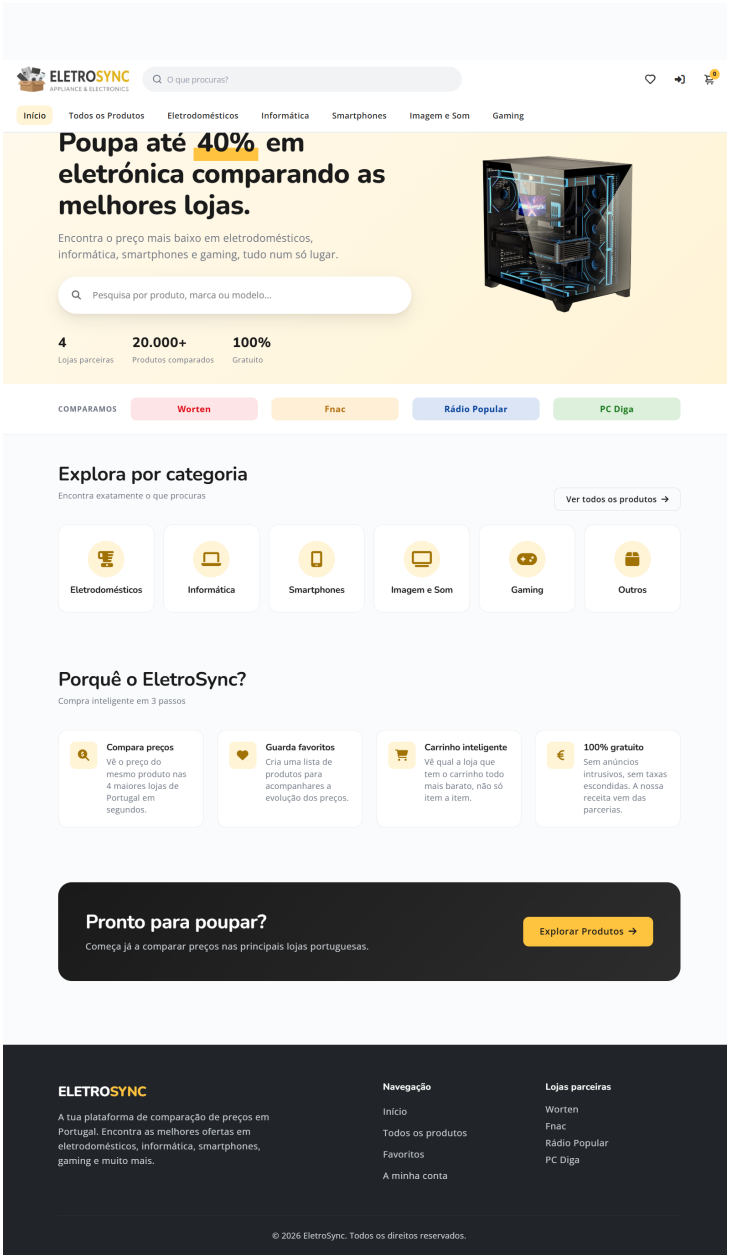


Figura 5: Página Inicial e Motor de Pesquisa

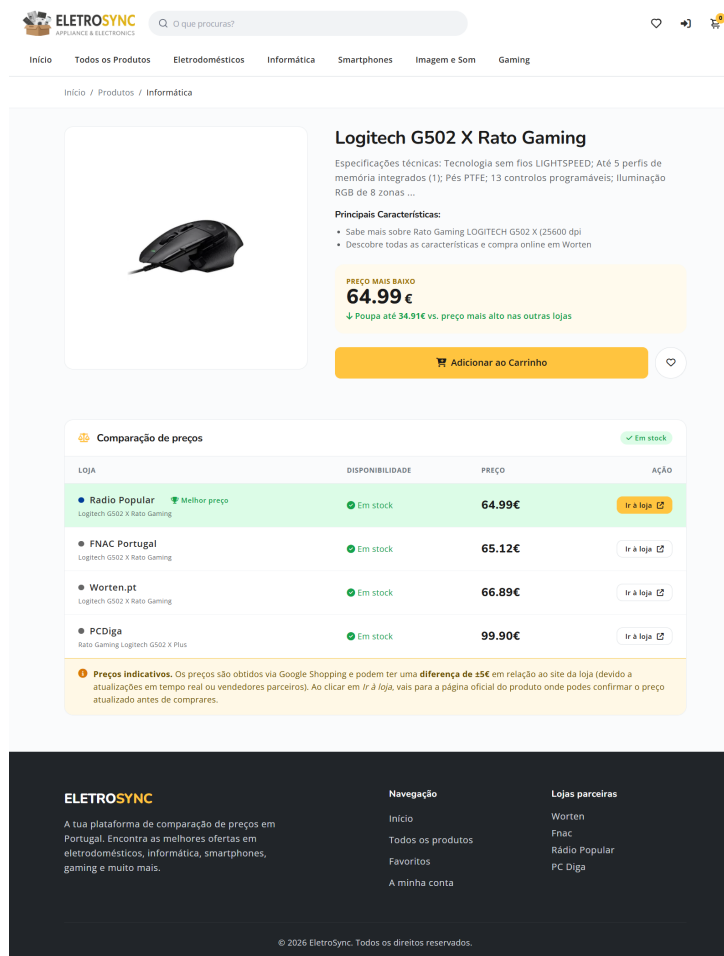


Figura 7: Área Dedicada

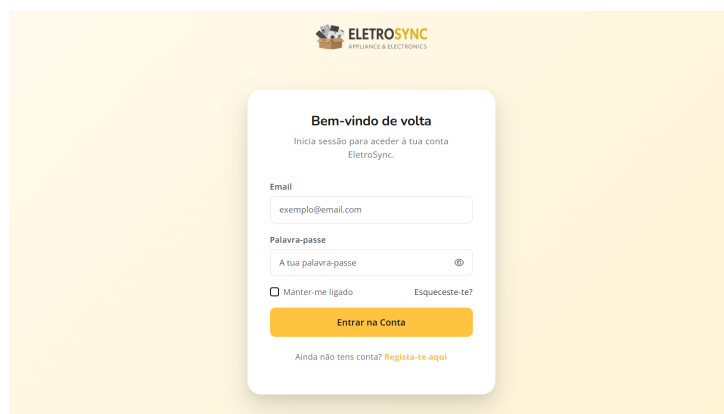


Figura 8: Login

